BỘ GIÁO DỤC VÀ ĐÀO TẠO

ĐẠI HỌC CẦN THƠ

BÁO CÁO CHUYÊN ĐỀ

TÀI LIỆU HỌC TẬP VÀ TRẠNG THÁI NGHIÊN CỨU

KIẾN TRÚC ĐA THUÊ BAO CHO ỨNG DỤNG QUẢN LÝ CÔNG VIỆC

Mã số đề tài: THS2026-67

Nhóm nghiên cứu đề tài

Cần Thơ, tháng 10 năm 2026

Mục lục

Danh sách bảng **iii**

Danh sách hình vẽ **iv**

Lời nói đầu **v**

Quy ước về nguồn và mức độ bằng chứng v

Cấu trúc tài liệu vi

1 Mở đầu **1**

1.1 Bối cảnh và bài toán nghiên cứu 1

1.2 Mục tiêu, đối tượng và phạm vi 1

1.3 Cách tiếp cận và yêu cầu về bằng chứng 2

1.4 Câu hỏi nghiên cứu và sự nhất quán về phạm vi 2

2 Cơ sở lý thuyết **4**

2.1 Thuê bao, dịch vụ, thể hiện và bản triển khai 4

2.2 Đa người dùng, đa thể hiện và đa thuê bao 4

2.3 Thứ bậc của PostgreSQL 5

2.4 Ba chiều cô lập 5

2.5 Ngăn xếp tổng thể và hai mặt phẳng 6

2.6 Vai trò của các công nghệ trong hiện thực hệ thống 7

3 Phân tích và lựa chọn kiến trúc **9**

3.1 Phương pháp xây dựng các kịch bản kiến trúc 9

3.2 Phân tích đánh đổi giữa các phương án 9

3.3 Cơ sở và giới hạn của phương án đề xuất S3 10

3.4 Đối chiếu phạm vi, hiện thực và kiến trúc đề xuất 11

3.5 Cơ chế quản lý kết nối trong hiện thực lịch sử 11

3.6 Thiết kế nghiệp vụ phục vụ kiểm chứng kiến trúc 12

4 Các kỹ thuật xây dựng kiến trúc **13**

4.1 Luồng xử lý yêu cầu và xác minh ngữ cảnh thuê bao 13

4.2 Bảo vệ tầng dữ liệu 13

4.3 Nhất quán khi nhiều người cùng sửa 14

4.4 Cấp phát và tính lặp an toàn 14
4.5 Outbox, tác vụ nền và thông báo 15
4.6 Tệp, đo lường và định giá 15
4.7 Cập nhật, khôi phục và phạm vi sự cố 16

5 Thực nghiệm, đánh giá và trạng thái bằng chứng 17
5.1 Phạm vi hiện vật và phiên bản được khảo sát 17
5.2 Bảng trạng thái theo loại công việc 17
5.3 Đối chiếu kết quả kiểm thử theo từng thời điểm 18
5.4 Thiết kế phép đo và yêu cầu tái lập 19
5.5 Bộ thí nghiệm được đề xuất 19
5.6 Định nghĩa và cách diễn giải các chỉ số 20
5.7 Mối đe dọa tính hợp lệ 21

6 Kết luận hiện tại và kế hoạch hoàn thiện 22
6.1 Kết quả tổng hợp và giới hạn kết luận 22
6.2 Các vấn đề cần xác nhận 22
6.3 Lộ trình hoàn thiện và tiêu chí đầu ra 23
6.4 Tổ chức chuyển giao kiến thức trong nhóm 23
6.5 Đóng góp có thể hướng tới và giới hạn 24

Phụ lục A. Câu hỏi ôn tập và gợi ý trả lời 25
Câu hỏi Chương 1 25
Câu hỏi Chương 2 25
Câu hỏi Chương 3 25
Câu hỏi Chương 4 25
Câu hỏi Chương 5 25
Gợi ý trả lời 26
Tình huống vận dụng 26

Phụ lục B. Danh mục nguồn và truy vết bằng chứng 28
Nguồn hiện có trong repository 28
Nguồn lịch sử Git 28

Danh sách bảng

1 Tổng hợp: Quy ước về nguồn và mức độ bằng chứng v

1.1 Tổng hợp: Mục tiêu, đối tượng và phạm vi 1

2.1 Tổng hợp: Đa người dùng, đa thể hiện và đa thuê bao 4
2.2 Tổng hợp: Thứ bậc của PostgreSQL 5
2.3 Tổng hợp: Ngăn xếp tổng thể và hai mặt phẳng 6
2.4 Tổng hợp: Vai trò của các công nghệ trong hiện thực hệ thống 7

3.1 Tổng hợp: Phương pháp xây dựng các kịch bản kiến trúc 9
3.2 Tổng hợp: Phân tích đánh đổi giữa các phương án 10
3.3 Tổng hợp: Đối chiếu phạm vi, hiện thực và kiến trúc đề xuất 11

5.1 Tổng hợp: Bảng trạng thái theo loại công việc 17
5.2 Tổng hợp: Đối chiếu kết quả kiểm thử theo từng thời điểm 18
5.3 Tổng hợp: Bộ thí nghiệm được đề xuất 19
5.4 Tổng hợp: Định nghĩa và cách diễn giải các chỉ số 20

6.1 Tổng hợp: Các vấn đề cần xác nhận 22
6.2 Tổng hợp: Tổ chức chuyển giao kiến thức trong nhóm 23
6.3 Tổng hợp: Nguồn hiện có trong repository 28
6.4 Tổng hợp: Nguồn lịch sử Git 29

Danh sách hình vẽ

2.1 Ba cách đặt cùng một loại dữ liệu của hai thuê bao. Đây là sơ đồ giải thích, không phải ảnh chụp hệ thống đang chạy. 6

Lời nói đầu

Tên đề tài: Xây dựng ứng dụng quản lý công việc theo kiến trúc đa thuê bao. Tài liệu theo sáu chương đề cương trong `resource/Research_Report.pdf`, bổ sung kiến thức nền, ví dụ, cơ chế, bằng chứng tiến độ và câu hỏi tự kiểm. Đối tượng sử dụng là các thành viên đã có kiến thức cơ bản về lập trình web và cơ sở dữ liệu, cần xây dựng hiểu biết có hệ thống về kiến trúc đa thuê bao.

Báo cáo chuyên đề được biên soạn nhằm hệ thống hóa kiến thức và hỗ trợ các thành viên nắm rõ trạng thái đề tài. Giá trị của các kết luận được giới hạn bởi bằng chứng đã thu thập; việc biên soạn không thay thế thủ tục nghiệm thu hoặc quyết định kiến trúc của nhóm. Thuyết minh gốc, phần mở rộng chưa duyệt, ứng dụng lịch sử và đề xuất mới được phân biệt rõ. Những vấn đề thiếu dữ liệu được giữ nguyên.

Tại thời điểm đối chiếu nguồn, nhóm đã có thuyết minh, ứng dụng ba placement trong lịch sử Git, biên bản kiểm thử local và phân tích kiến trúc mới. Checkout hiện không chứa mã ứng dụng; đề xuất S3 còn cần nhóm xác nhận; các nguồn đã kiểm tra chưa cung cấp bộ bằng chứng thực nghiệm chính thức đủ để kết luận kiến trúc nào tối ưu.

Quy ước về nguồn và mức độ bằng chứng

Bảng 1: Tổng hợp: Quy ước về nguồn và mức độ bằng chứng

Nhãn	Ý nghĩa trong tài liệu
[HIỆN CÓ]	File hoặc nội dung được kiểm tra trực tiếp trong checkout ngày 23/10/2026.
[MÃ LỊCH SỬ]	Đã đọc mã tại commit được chỉ rõ; chưa chạy lại mã đó.
[BIÊN BẢN CŨ]	Kết quả do tài liệu của lần kiểm tra trước ghi nhận; không phải lần chạy mới.
[NGUỒN]	Nội dung có căn cứ từ tài liệu kỹ thuật chính thức; nguồn được đặt cạnh phần giải thích.
[DIỄN GIẢI]	Ví dụ hoặc suy luận dùng để giải thích; không phải kết quả đo.
[ĐỀ XUẤT]	Phương án, giả định hoặc ngưỡng chưa thay thế quyết định đã duyệt.

Nhãn	Ý nghĩa trong tài liệu
[CHƯA CÓ]	Chưa xác nhận được bằng chứng cần thiết trong phạm vi kiểm tra này.

Mức độ hoàn thành được đánh giá theo bốn khía cạnh: chức năng ứng dụng, cơ sở kiến trúc, bằng chứng thực nghiệm và hồ sơ bàn giao. Một tỷ lệ hoàn thành chung không thể hiện đầy đủ sự khác biệt giữa các khía cạnh này.

Cấu trúc tài liệu

Báo cáo được tổ chức thành sáu chương. Chương 1 trình bày bài toán, mục tiêu và phạm vi. Chương 2 hệ thống hóa cơ sở lý thuyết. Chương 3 phân tích và lựa chọn kiến trúc; Chương 4 giải thích các cơ chế hiện thực. Chương 5 đối chiếu bằng chứng kiểm thử và thiết kế thực nghiệm. Chương 6 tổng hợp kết quả, giới hạn và lộ trình hoàn thiện. Phụ lục A cung cấp câu hỏi ôn tập và tình huống vận dụng; Phụ lục B ghi nguồn và cách truy vết.

Chương 1

Mở đầu

1.1 Bối cảnh và bài toán nghiên cứu

Giả sử trường có hai nhóm cần quản lý công việc: **nhóm A — lớp học phần** và **nhóm B — phòng thí nghiệm**. Mỗi nhóm có thành viên, dự án, bảng Kanban, công việc và tệp riêng. Kanban là cách biểu diễn công việc thành thẻ nằm trong các cột trạng thái, chẳng hạn "Cần làm", "Đang làm", "Hoàn tất". Di chuyển thẻ thể hiện thay đổi trạng thái công việc.

Nếu cài một ứng dụng hoàn toàn riêng cho mỗi nhóm, người vận hành phải lặp lại cập nhật, sao lưu và cấu hình. Nếu đưa tất cả vào một ứng dụng chung mà không có ranh giới dữ liệu, nhóm A có thể đọc hoặc sửa việc của nhóm B. **Bài toán nghiên cứu là chia sẻ tài nguyên để vận hành thuận tiện, đồng thời giữ đúng ranh giới và quyền của từng nhóm.**

Trong đề tài, một **không gian làm việc — workspace** được chọn làm một **thuê bao — tenant**. Một workspace chứa nhiều dự án. Vì vậy, một dự án không tự động là một thuê bao. Ví dụ này là minh họa nghiệp vụ, không phải dữ liệu về hai đơn vị đã tham gia thử nghiệm. [DIỄN GIẢI; R01, mục 13]

Một người tên Lan có thể ở cả A và B: là người quản lý dự án tại A nhưng chỉ được xem dự án tại B. Khi Lan đổi workspace, dữ liệu và quyền phải đổi theo. Chính trường hợp này cho thấy "đăng nhập thành công" chưa đủ để xử lý yêu cầu an toàn.

1.2 Mục tiêu, đối tượng và phạm vi

Thuyết minh đặt ba mục tiêu: phân tích các cơ chế đa thuê bao; xây dựng ứng dụng áp dụng kiến trúc đề xuất và triển khai qua Internet; thực nghiệm ở quy mô vừa phải để đánh giá tính khả thi. Các cơ chế trọng tâm là định danh thuê bao, cô lập dữ liệu, phân quyền và nhất quán trạng thái công việc. [HIỆN CÓ; R01, mục 12–14]

Bảng 1.1: Tổng hợp: Mục tiêu, đối tượng và phạm vi

Phần	Phạm vi và cách hiểu
Đối tượng nghiên cứu	Kiến trúc đa thuê bao gắn với nghiệp vụ quản lý công việc.
Phương tiện kiểm chứng	Ứng dụng Kanban: tạo tình huống đọc, ghi, phân quyền, tệp và tác vụ nền.

Phần	Phạm vi và cách hiểu
Quy mô thực nghiệm gốc	3–5 thuê bao giả lập, mỗi thuê bao khoảng 10–20 người dùng hoạt động đồng thời.
Mô hình dữ liệu trong phạm vi gốc	Chung CSDL–chung lược đồ và chung CSDL–riêng lược đồ.
Phần mở rộng	CSDL riêng, Bridge ba placement và các mức tùy biến trong Phụ lục A đang ghi chờ phê duyệt học thuật.
Ngoài trọng tâm	Mobile, đồng bộ thời gian thực phức tạp, tối ưu hạ tầng quy mô lớn và tự động mở rộng.

Điểm cần nhóm xác nhận: mục 5 của thuyết minh ghi thời gian 5–10/2026, nhưng mục 13 và bảng tiến độ vẫn ghi 3–8/2026. Tài liệu học giữ nguyên sự khác biệt này để nhóm đối chiếu hồ sơ đã ký; không tự xác lập thời hạn chính thức. Tương tự, mốc thủ tục trong biên bản họp là ghi nhận tại buổi họp, không chứng minh thủ tục đã hoàn tất. [HIỆN CÓ; R01, R03]

1.3 Cách tiếp cận và yêu cầu về bằng chứng

"Có trang tạo công việc" chứng minh một chức năng được hiện thực. "Mọi đường đọc/ghi đều bị giới hạn bởi thuê bao và quyền dự án" là một thuộc tính kiến trúc cần bằng chứng rộng hơn. "S3 tốt hơn S4 trong điều kiện X" lại cần phép so sánh có kiểm soát. Ba phát biểu đòi hỏi ba loại bằng chứng khác nhau.

Chuỗi lập luận cần giữ xuyên suốt là: **nhu cầu → yêu cầu → phương án kiến trúc → cơ chế cài đặt → phép kiểm chứng → kết luận có giới hạn**. Ví dụ, yêu cầu không đọc chéo dẫn đến kiểm tra ngữ cảnh, quyền dự án và chính sách CSDL; bằng chứng là những lần gọi hợp lệ và trái phép đã chạy, không phải ảnh trang đăng nhập.

Theo ghi chú họp, giảng viên yêu cầu đi từ khái niệm và mô hình tổng thể tới kịch bản, đánh đổi rồi lựa chọn kiến trúc. Ứng dụng đóng vai trò minh chứng cho lựa chọn đó. Đây là định hướng được biên bản ghi lại, không phải khẳng định toàn bộ nội dung mới đã được giảng viên duyệt. [HIỆN CÓ; R03]

1.4 Câu hỏi nghiên cứu và sự nhất quán về phạm vi

Protocol lịch sử phiên bản 0.1 có RQ1 về yêu cầu, RQ2 về Bridge Pool/Silo, RQ3 về mức đáp ứng các thuộc tính và RQ4 về cơ chế trên VPS hạn chế. Bộ nghiên cứu mới

hướng tới chọn một kiến trúc chính S3. Hai cách đặt vấn đề chưa tự động tương đương. [BIÊN BẢN CŨ; H03. HIỆN CÓ; R05]

[ĐỀ XUẤT] Khi nhóm chốt hướng mới, có thể diễn đạt lại câu hỏi trung tâm: "Trong phạm vi ứng dụng và nguồn lực đã chọn, nên chia sẻ hay tách riêng những tầng nào, dùng cơ chế nào để bảo vệ ranh giới thuê bao, và bằng chứng nào cho thấy các đánh đổi là chấp nhận được?" Cần ghi sự thay đổi vào hồ sơ phương pháp trước khi thu số liệu chính thức; tài liệu này không sửa protocol cũ.

Chương 2

Cơ sở lý thuyết

2.1 Thuê bao, dịch vụ, thể hiện và bản triển khai

Dịch vụ là chức năng người dùng gọi qua giao diện xác định; ví dụ tạo công việc qua ứng dụng web. **Nhà cung cấp** vận hành dịch vụ. **Khách hàng** là bên sử dụng dịch vụ theo quan hệ với nhà cung cấp. **Người dùng** là cá nhân hoặc tác nhân sử dụng dịch vụ. **Thuê bao** là đơn vị được hệ thống quản lý với dữ liệu, cấu hình và quyền riêng. Trong đề tài, đơn vị đó là workspace. Một khách hàng có thể có nhiều thuê bao và một thuê bao có nhiều người dùng. [DIỄN GIẢI theo R04; Azure: tenancy models]

Thể hiện — instance là một bản đang chạy của một thành phần: tiến trình API, máy chủ CSDL. Luôn nói rõ thể hiện của thành phần nào. **Bản triển khai — deployment** là tập các thành phần được triển khai và vận hành cùng nhau. Một bản triển khai có thể phục vụ nhiều thuê bao. Việc chạy hai bản sao API để cân bằng tải không có nghĩa đã tạo hai thuê bao. [DIỄN GIẢI; R04]

Chia sẻ nghĩa là cùng một thể hiện phục vụ nhiều thuê bao. **Dùng riêng** nghĩa là thể hiện đó dành cho một thuê bao. Cách kiểm tra chắc chắn là liệt kê thuê bao được phục vụ bởi mỗi thể hiện. Câu hỏi "thêm thuê bao có phải tạo thể hiện không?" chỉ là gợi ý: hệ thống có thể đã cấp phát sẵn thể hiện dự phòng, hoặc cần thêm bản sao chung vì quá tải. Không dùng gợi ý đó làm định nghĩa tuyệt đối.

2.2 Đa người dùng, đa thể hiện và đa thuê bao

Bảng 2.1: Tổng hợp: Đa người dùng, đa thể hiện và đa thuê bao

Khái niệm	Ví dụ và điểm cần nhận ra
Đa người dùng	Nhiều người cùng dùng phần mềm; có thể có phân quyền và dữ liệu riêng. Chỉ biết có nhiều tài khoản chưa đủ để biết ranh giới tổ chức.
Đa thể hiện	Có nhiều bản đang chạy của một thành phần; có thể để chia tải hoặc phục vụ những thuê bao khác nhau.

Đa thuê bao	Hệ thống phục vụ nhiều đơn vị thuê bao, quản lý ranh giới dữ liệu/quyền/cấu hình, có tài nguyên được chia sẻ ở một hay nhiều tầng.

Các khái niệm không loại trừ nhau. Một SaaS đa thuê bao có nhiều người dùng và nhiều bản sao API vẫn phù hợp. Cần tránh cách diễn giải rằng "multi-user luôn dùng chung toàn bộ dữ liệu" hoặc "multi-tenant chỉ được chạy một tiến trình". Tài liệu R04 trình bày phân loại của Kabbedijk; đó là một khung học thuật cụ thể, cần giữ ngữ cảnh khi sử dụng.

2.3 Thứ bậc của PostgreSQL

Đọc từ ngoài vào trong: **cluster PostgreSQL → database → schema → table → row**. Trong ngữ cảnh PostgreSQL, cluster ở đây là tập CSDL do một hệ thống máy chủ PostgreSQL quản lý; không nhất thiết là cụm nhiều máy. Schema là không gian tên chứa bảng và các đối tượng khác, không phải một máy chủ. [NGUỒN; PostgreSQL: Managing Databases]

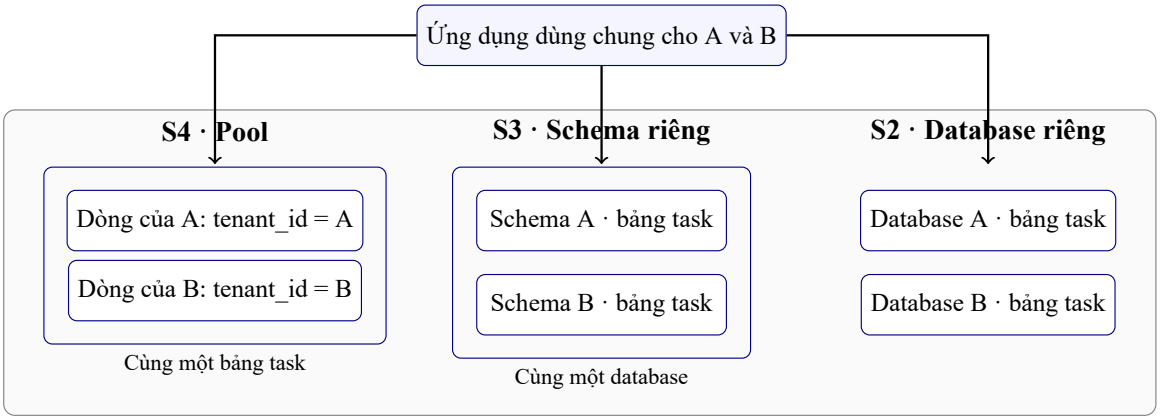
Bảng 2.2: Tổng hợp: Thứ bậc của PostgreSQL

Mô hình	Dữ liệu công việc của A và B nằm ở đâu?
Pool	Cùng bảng `task`; dòng mang `tenant_id=A` hoặc `tenant_id=B`.
Schema-per-tenant	Trong một database; A dùng `tenant_a.task`, B dùng `tenant_b.task`.
Silo database	A dùng database A, B dùng database B; mỗi database có bảng `task`.

Bridge là cách phối hợp các mô hình cô lập, chẳng hạn A ở Pool, B ở Schema-per-tenant, C ở Silo. Nó không phải tên khác của Schema-per-tenant. Đây cũng là cách Phụ lục A và ứng dụng lịch sử dùng thuật ngữ. Sổ nguồn R07 ghi nhận cách gọi của AWS; phiên tổng hợp này chưa mở lại được trang AWS, nên không nâng việc dẫn lại đó thành một xác minh trực tuyến mới.

2.4 Ba chiều cô lập

Cô lập dữ liệu: A không đọc, sửa hay xóa dữ liệu của B. **Cô lập hiệu năng:** tải của A được kiểm soát để không gây suy giảm quá mức cho B. **Cô lập sự cố và vận hành:** lỗi



Ba phương án so sánh: đều có thể dùng chung máy chủ PostgreSQL.
Tách dữ liệu không tự tách CPU, RAM và đĩa.

Hình 2.1: Ba cách đặt cùng một loại dữ liệu của hai thuê bao. Đây là sơ đồ giải thích, không phải ảnh chụp hệ thống đang chạy.

hoặc thao tác cập nhật/khôi phục của A có phạm vi ảnh hưởng được quản lý.

Ví dụ, tách database A và B giúp thiết lập quyền truy cập riêng, nhưng cả hai vẫn có thể tranh CPU, bộ nhớ và đĩa trên cùng máy. Vì vậy, "Silo database" không đồng nghĩa với "không có hàng xóm ồn ào". Tách container cũng cần xem tài nguyên máy chủ bên dưới có chung hay không. Đây là suy luận kiến trúc từ các tầng tài nguyên chung, không phải phép đo trên ứng dụng. [DIỄN GIẢI; R06; nguồn PostgreSQL ở trên]

Bán kính ảnh hưởng là phạm vi thuê bao chịu tác động của một lỗi. Sai quyền trên schema A có thể có phạm vi khác với chết tiến trình API chung; lộ tài khoản cấp phát có quyền quản trị có thể ảnh hưởng rộng hơn lộ một tài khoản runtime của A. Việc đánh giá cần nêu lỗi cụ thể, quyền của tác nhân và tầng xảy ra lỗi.

2.5 Ngăn xếp tổng thể và hai mặt phẳng

Bảng 2.3: Tổng hợp: Ngăn xếp tổng thể và hai mặt phẳng

Thành phần	Trách nhiệm trong ví dụ A/B
Trình duyệt và giao diện	Hiển thị workspace đang chọn; gửi yêu cầu cùng phiên truy cập.
Biên truy cập	Nhận host, HTTPS và chuyển yêu cầu tới API.
Định danh, ngữ cảnh và phân quyền	Xác minh người gọi, thuê bao, tư cách thành viên và quyền thao tác.
Nghiệp vụ	Xử lý dự án, bảng, công việc; giữ các quy tắc trạng thái.
Dữ liệu	Lưu dữ liệu theo placement và thực thi các giới hạn truy cập.

Thành phần	Trách nhiệm trong ví dụ A/B
Tệp và tác vụ nền	Lưu đối tượng, xử lý sự kiện, thông báo; đều phải giữ phạm vi thuê bao.
Vận hành	Cấp phát, theo dõi, sao lưu, cập nhật và xử lý lỗi.

Mặt phẳng ứng dụng xử lý công việc người dùng: tạo task, xem bảng, tải tệp. **Mặt phẳng điều khiển** quản lý dịch vụ: tạo thuê bao, lưu placement, quản lý trạng thái và cấu hình. Nó phục vụ quản trị toàn hệ thống nhưng vẫn lưu và thao tác thông tin theo từng thuê bao; không được hiểu "toàn cục" là bỏ xác thực hay không biết `tenant_id`. [NGUỒN; Azure: control planes]

Ví dụ A đăng ký workspace: mặt phẳng điều khiển tạo và cấp phát A. Sau khi A sẵn sàng, người dùng tạo task qua mặt phẳng ứng dụng. Tách trách nhiệm giúp tránh để tiến trình phục vụ thao tác thông thường có toàn bộ quyền tạo/xóa CSDL. [DIỄN GIẢI; H01]

2.6 Vai trò của các công nghệ trong hiện thực hệ thống

Theo hồ sơ và mã lịch sử, backend dùng Java/Spring Boot, giao diện React/TypeScript, CSDL PostgreSQL, Flyway cho migration, HikariCP cho nhóm kết nối; các công cụ quan sát và kiểm thử gồm Micrometer, Prometheus, k6, Testcontainers và Playwright. Đây là mô tả phiên bản ứng dụng đã lưu, không phải danh sách phụ thuộc được xác nhận đang cài trong checkout tại thời điểm khảo sát. [BIÊN BẢN CŨ; H01]

Bảng 2.4: Tổng hợp: Vai trò của các công nghệ trong hiện thực hệ thống

Công cụ/khái niệm	Giải thích cho người mới
Migration	Tệp thay đổi cấu trúc dữ liệu có phiên bản; giúp biết một schema đã nâng đến đâu.
Nhóm kết nối	Giữ và tái sử dụng kết nối CSDL; tiết kiệm chi phí mở kết nối nhưng phải quản lý trạng thái theo yêu cầu.
Testcontainers	Tạo môi trường container cho test tích hợp; test có thể bị bỏ qua khi thiếu Docker.
k6	Sinh tải theo kịch bản; dữ liệu sinh tải chưa tự động là dữ liệu nghiên cứu hợp lệ.
Prometheus	Thu thập chuỗi số đo; dashboard hiển thị số đo, không thay dữ liệu thô và cấu hình chạy.

Công cụ/khái niệm	Giải thích cho người mới
Playwright	Điều khiển trình duyệt để kiểm tra luồng người dùng.

Chương 3

Phân tích và lựa chọn kiến trúc

3.1 Phương pháp xây dựng các kịch bản kiến trúc

Thay vì bắt đầu bằng "dùng Pool hay Silo", cần xác định quyết định chia sẻ/dùng riêng trên từng tầng. Bộ nghiên cứu hiện tại cố định ứng dụng dùng chung và thay đổi cách đặt dữ liệu. Đây là cách thu hẹp không gian để so sánh được, không phải tuyên bố kiến trúc đa thuê bao chỉ có tầng CSDL. [HIỆN CÓ; R04–R06]

Bảng 3.1: Tổng hợp: Phương pháp xây dựng các kịch bản kiến trúc

Kịch bản	Nội dung và vai trò
S0	Ứng dụng và máy chủ CSDL riêng theo thuê bao; điểm tham chiếu.
S1	Ứng dụng chung, máy chủ CSDL riêng theo thuê bao; đối chứng tùy chọn.
S2	Ứng dụng chung, database riêng trên máy chủ CSDL chung; `SILO_DATABASE`.
S3	Ứng dụng chung, database chung, schema riêng; `SCHEMA_PER_TENANT`.
S4	Ứng dụng chung, database/schema chung, phân biệt bằng `tenant_id`; `POOL`.
S5	Chính sách dùng phối hợp S2/S3/S4 theo thuê bao; Bridge.

Các mã là quy ước của bộ nghiên cứu này. Trong ký hiệu của Kabbedijk, S3 được viết là $\langle AI, DB \rangle$; chữ DB ở đây là mã một mức của trục dữ liệu, không nên tự suy ra nghĩa chỉ từ viết tắt. Đọc bảng diễn giải trước khi đọc ký hiệu. [R04–R06]

3.2 Phân tích đánh đổi giữa các phương án

Bảng 3.2: Tổng hợp: Phân tích đánh đổi giữa các phương án

Câu hỏi	So sánh S2, S3 và S4
Cấp phát thêm A cần làm gì?	S2 tạo database; S3 tạo schema; S4 thường thêm dữ liệu quản lý. Thời gian thực phải đo.
Nâng cấu trúc bảng cần bao nhiêu nơi?	S2/S3 có nhiều database/schema cần theo dõi; S4 thay cấu trúc chung. Một lần thay đổi chung vẫn có thể ảnh hưởng nhiều thuê bao.
Bảo vệ ở CSDL bằng gì?	S2 quyền database/tài khoản; S3 quyền schema/tài khoản; S4 chính sách mức dòng cùng ngữ cảnh đúng.
A tải quá nặng thì B có chậm không?	Cả ba có tài nguyên chung; cần đo và kiểm soát tải.
Khôi phục riêng A dễ đến đâu?	Phải xét dữ liệu, phụ thuộc, quyền và tệp; tách database/schema tạo đơn vị thao tác thuận tiện hơn nhưng vẫn phải thử khôi phục.
Có cần ba placement trong sản phẩm?	Chỉ khi lợi ích theo yêu cầu bù được công sức vận hành, kiểm thử và hỗ trợ nhiều nhánh.

An toàn là điều kiện loại. Một phương án để lộ dữ liệu không thể được chấp nhận chỉ vì có điểm chi phí hoặc hiệu năng cao. Ngược lại, không có truy cập trái phép trong vài test cũng chưa chứng minh an toàn trước mọi kiểu tấn công. Phải nêu phạm vi các ca đã chạy. [DIỄN GIẢI; H03, R06]

3.3 Cơ sở và giới hạn của phương án đề xuất S3

`research/05-lua-chon-kien-truc.md` đề xuất S3 làm kiến trúc chính, S2/S4 làm đối chứng hoặc biến thể có điều kiện. Phân tích lấy điểm chuyên gia trong tài liệu Kabbedijk năm 2014, kết hợp trọng số và giả định quy mô của nhóm. Các kết quả như mức hối tiếc 0,03 hoặc tỷ lệ thắng khi biến thiên trọng số là **kết quả của mô hình chấm điểm**, không phải thời gian phản hồi, tỷ lệ an toàn hay xác suất triển khai thành công. Phiên tổng hợp này đọc kết quả đã lưu, chưa chạy lại script hay kiểm lại toàn bộ Bảng 6.3 từ luận án. [HIỆN CÓ; R05, R07]

Hối tiếc ở đây là khoảng hụt điểm so với ứng viên đứng đầu trong một hồ sơ yêu cầu. Chọn phương án có hối tiếc lớn nhất nhỏ nhất là cách hạn chế việc chọn quá tệ cho một nhóm yêu cầu. Ví dụ minh họa: nếu S3 hụt 0,1 điểm ở một hồ sơ và 0,4 ở hồ sơ khác, hối tiếc lớn nhất là 0,4. Các số trong ví dụ này chỉ để giải nghĩa phép tính.

Khi chỉ xét tập ứng viên S2–S4, phân tích hiện có đổi lựa chọn khi thay giả định quy mô: ở thiết lập hướng đến nhiều thuê bao nhỏ, S3 được ưu tiên; ở thiết lập quy mô nhỏ, S2 được ưu tiên. Nếu đưa cả S1 vào, thứ hạng thay đổi; phải ghi rõ tập ứng viên khi trích kết quả. Nhóm cần xác nhận quy mô thiết kế thay vì lấy quy mô của một lần demo làm yêu cầu toàn dịch vụ. Chưa có ánh xạ thực nghiệm chứng minh một hệ số trọng số tương ứng chính xác bao nhiêu thuê bao. [HIỆN CÓ; R05]

Đầu file kết quả phân tích vẫn ghi Kabbedijk "2015", trong khi sổ hiệu chỉnh R07 ghi luận án năm 2014. Tài liệu này theo năm 2014 mà sổ nguồn đã hiệu chỉnh, đồng thời giữ dấu vết sự chưa đồng bộ để nhóm rà trích dẫn; không sửa file kết quả của lần nghiên cứu trước.

3.4 Đối chiếu phạm vi, hiện thực và kiến trúc đề xuất

Bảng 3.3: Tổng hợp: Đối chiếu phạm vi, hiện thực và kiến trúc đề xuất

Lớp hồ sơ	Trạng thái được xác nhận
Thuyết minh gốc	Nêu nghiên cứu chung schema/riêng schema; Phụ lục A thêm Bridge ba placement và đang chờ phê duyệt.
Ứng dụng lịch sử	Đã hiện thực nhánh Pool, Schema-per-tenant và Silo; có biên bản local.
Đề xuất mới	S3 là kiến trúc chính theo phân tích; danh sách cần nhóm xác nhận vẫn tồn tại trong README nghiên cứu.

Vì vậy không viết "nhóm đã chính thức chuyển sang S3" hay "Bridge đã được chứng minh tối ưu". Phát biểu đúng hiện tại là: **đã có ứng dụng hỗ trợ ba placement trong lịch sử; tài liệu nghiên cứu mới đề xuất S3; quyết định học thuật và kiểm chứng định lượng còn cần hoàn thiện**.

3.5 Cơ chế quản lý kết nối trong hiện thực lịch sử

Trong `DefaultTenantDataSourceResolver.resolve`, nhánh Pool dùng một datasource chung. Hai placement còn lại lấy datasource từ bộ nhớ đệm khóa theo `tenantId`; khi tạo mới, mã dùng username và mật khẩu của placement. `schemaName` chọn schema riêng cho Schema-per-tenant. [MÃ LỊCH SỬ; H04]

Như vậy, **S3 của bản mã đã đọc dùng nhóm kết nối theo thuê bao**, không phải một nhóm chung rồi chuyển vai trò bằng `SET ROLE`. `TenantJdbcExecutor` đặt `app.tenant_id`,

`app.user_id` và `search_path` trong phạm vi giao dịch; đoạn mã đó không thực hiện chuyển role. [MÃ LỊCH SỬ; H04–H05]

Hệ quả: không dùng lập luận "S3 tránh phân mảnh nhóm kết nối còn S2 không tránh được" để mô tả bản ứng dụng này. Muốn dùng cách kết nối khác phải coi đó là thay đổi thiết kế và kiểm chứng lại. Có tài khoản riêng cũng không đủ để khẳng định an toàn trước mọi SQL injection: còn tùy đặc quyền được cấp, cấu hình và việc ứng dụng có thể chọn nhầm tài khoản hay không.

Đây là cập nhật bằng chứng cho câu hỏi còn mở trong tài liệu 08, không phải thay đổi mã hay sửa kết luận của bộ nghiên cứu trước.

3.6 Thiết kế nghiệp vụ phục vụ kiểm chứng kiến trúc

Quan hệ chính là: workspace chứa dự án; dự án chứa bảng; bảng có cột và công việc; công việc có người được giao, bình luận, tệp và trạng thái. Vai trò cấp workspace và vai trò cấp dự án giải quyết hai phạm vi khác nhau. Có quyền quản lý thành viên workspace không nên được suy diễn thành quyền vô hạn ở mọi API nghiệp vụ nếu chưa đọc chính sách tương ứng. [DIỄN GIẢI; H01]

Những tình huống có giá trị nghiên cứu gồm: Lan thuộc A/B nhưng quyền khác nhau; xóa thành viên khi đang có phiên hoạt động; hai người sửa một task; xóa tệp khi worker bị lỗi; cấp phát bị dừng giữa chừng. Chúng buộc cơ chế đa thuê bao thể hiện trên toàn luồng, vượt ra ngoài truy vấn đọc danh sách.

Chương 4

Các kỹ thuật xây dựng kiến trúc

4.1 Luồng xử lý yêu cầu và xác minh ngữ cảnh thuê bao

Ví dụ Lan mở bảng của A qua host `a.example.test`, nhưng thay ID công việc trong URL bằng ID thuộc B. Tên miền và ID ở đây chỉ minh họa. Một chuỗi xử lý cần trả lời liên tiếp:

1. **Ai gọi?** Xác thực phiên/token. Token là thông tin xác thực do hệ thống cấp; không tin nội dung chỉ vì trình duyệt gửi lên.
2. **Gọi vào thuê bao nào?** Đối chiếu host với thuê bao trong token và bản ghi hệ thống.
3. **Còn là thành viên không?** Kiểm tra membership đang hoạt động và phiên quyền có còn hợp lệ.
4. **Được thao tác gì?** Kiểm tra vai trò, dự án, trạng thái tài nguyên và hành động yêu cầu.
5. **Dùng dữ liệu ở đâu?** Resolver chọn placement từ thông tin hệ thống đã xác minh.
6. **CSDL có cho phép không?** Thực thi quyền/chính sách tương ứng, trong giao dịch mang ngữ cảnh đúng.
7. **Sau yêu cầu còn sót ngữ cảnh không?** Dọn ngữ cảnh luồng xử lý, kết thúc giao dịch và trả kết nối đúng cách.

`TenantContextFilter` lịch sử đối chiếu host–token, tenant đang `ACTIVE`, membership và `membership_version`, sau đó tạo ngữ cảnh; khối `finally` xóa ngữ cảnh. Đây là bằng chứng đã đọc cơ chế trong mã, không phải một lần thử tấn công mới. [MÃ LỊCH SỬ; H06]

IDOR là tình huống thay định danh đối tượng để truy cập đối tượng không được phép. Chặn host/token lệch chỉ là một lớp: khi token và host cùng A, API vẫn phải từ chối ID công việc của B và công việc thuộc dự án mà Lan không có quyền.

4.2 Bảo vệ tầng dữ liệu

Ở Pool, bảng có nhiều tenant nên truy vấn và chính sách phải giữ ranh giới dòng. **RLS — Row-Level Security** cho phép CSDL giới hạn dòng được đọc hoặc thay đổi. Khi bật RLS mà không có chính sách áp dụng, hành vi mặc định là từ chối truy cập dòng.

Superuser và role `BYPASSRLS` vẫn vượt RLS; `FORCE ROW LEVEL SECURITY` áp dụng cho chủ bảng, không biến superuser thành tài khoản bị giới hạn. [NGUỒN; PostgreSQL: Row Security Policies]

Ở Schema-per-tenant, schema tạo không gian tên; **quyền mới quyết định ai truy cập được**. Đặt `search_path` để chọn schema thuận tiện không thay thế việc thu hẹp quyền. Tài khoản có quyền trên cả A/B vẫn có thể gọi tên đầy đủ `tenant_b.task`. Cần kiểm cả truy vấn ghi rõ schema khác. [NGUỒN; PostgreSQL: Schemas]

Ở Silo database, kết nối chọn một database, nhưng cần cấu hình quyền kết nối và quyền đối tượng phù hợp. Tách database không tự bảo đảm tài khoản A không thể mở kết nối khác tới B. Cơ chế kết nối liên CSDL cũng phải nằm trong mô hình đe dọa nếu được bật. [NGUỒN; PostgreSQL Managing Databases, mục 22.1]

Ngữ cảnh theo giao dịch: khi một kết nối được tái sử dụng, trạng thái của A không được lọt sang yêu cầu B. `TenantJdbcExecutor` dùng `set_config(..., true)` trong giao dịch, commit/rollback rồi hoàn trả trạng thái autocommit. Đây là cơ chế được quan sát trong mã; phải test cả đường thất bại và tái sử dụng kết nối. [MÃ LỊCH SỬ; H05]

4.3 Nhất quán khi nhiều người cùng sửa

Giả sử Lan và Minh cùng đọc task có phiên bản 7. Lan đổi hạn và lưu thành phiên bản 8. Minh dùng dữ liệu cũ để đổi người được giao. Nếu ghi đè toàn bộ bản cũ, thay đổi của Lan có thể mất. **Kiểm soát xung đột bằng phiên bản — optimistic concurrency** yêu cầu phiên bản khi ghi khớp phiên bản đang lưu; nếu không thì từ chối và yêu cầu nạp lại/giải quyết xung đột. [DIỄN GIẢI; cơ chế được H01 mô tả]

Ví dụ này là mô hình học, không khẳng định mọi endpoint lịch sử đã áp dụng đúng cùng một câu SQL. Kiểm chứng cần có hai thao tác cạnh tranh thật, kiểm dữ liệu cuối cùng và phản hồi lỗi. HTTP thành công ở một lượt ghi không chứng minh chống mất cập nhật.

4.4 Cấp phát và tính lặp an toàn

Cấp phát — provisioning biến một yêu cầu đăng ký thành tài nguyên sẵn sàng: tạo metadata, database/schema và role khi cần, chạy migration, chuẩn bị dữ liệu rồi đánh dấu hoạt động. Nếu đánh dấu `ACTIVE` quá sớm, người dùng có thể được chuyển vào một nơi chưa truy cập được.

Idempotency — lặp an toàn nghĩa là thử lại cùng yêu cầu không tạo thêm kết quả nghiệp vụ ngoài ý muốn. Ví dụ, callback thanh toán đến hai lần không được tạo

hai workspace. **Retry** là thử lại; **rollback/compensation** là hoàn tác hoặc bù cho phần đã làm khi không tiếp tục được. Chúng không đồng nghĩa: retry sai có thể nhân đôi tài nguyên; rollback cũng có thể thất bại và cần trạng thái lỗi riêng. [DIỄN GIẢI; H01]

Biên bản 08/09 ghi đã sửa lỗi bỏ sót `schema_name` khi sao chép placement, khiến thuê bao có thể bị đánh dấu hoạt động nhưng resolver từ chối. Đây là ví dụ cụ thể cho sự khác biệt giữa "có state machine" và "đã đi qua các chuyển trạng thái đúng". Việc khắc phục được ghi nhận theo biên bản ngày 08/09; lượt tổng hợp này không tái hiện hoặc chạy lại trường hợp lỗi. [BIÊN BẢN CŨ; H02]

4.5 Outbox, tác vụ nền và thông báo

Outbox là nơi lưu sự kiện cần xử lý tiếp cùng với thay đổi nghiệp vụ trong giao dịch CSDL. Ví dụ, ghi nhận task đã giao cho Lan và ghi sự kiện cần thông báo; worker đọc sự kiện để gửi sau. Cách này xử lý khoảng hở "đã lưu task nhưng tiến trình chết trước khi gửi thông báo". [DIỄN GIẢI; H01]

Worker có thể gửi xong rồi chết trước khi đánh dấu hoàn tất, nên sự kiện có thể được xử lý lại. Vì vậy cần cơ chế chống trùng ở hành động nhận, retry có giới hạn và theo dõi lỗi. Không nên gọi outbox là bảo đảm "gửi đúng một lần" cho mọi hệ thống bên ngoài.

Một job phải mang tenant và định danh tài nguyên, thiết lập ngữ cảnh phù hợp, kiểm lại quyền/trạng thái cần thiết rồi mới hành động. Nếu Lan đã rời A trước lúc worker chạy, thông báo không nên tự động dùng danh sách người nhận cũ. H01 ghi nhận các kiểm tra trạng thái này cho nhắc hạn/thông báo; mức đầy đủ vẫn cần đối chiếu từng loại sự kiện.

4.6 Tệp, đo lường và định giá

Tệp: metadata của tài nguyên và nội dung tệp có thể ở hai nơi khác nhau. Cần giới hạn truy cập cả hai. Khóa đối tượng gắn tenant phải do phía server kiểm soát; URL ký có thời hạn không thay thế việc kiểm quyền trước lúc cấp URL. Xóa metadata và xóa tệp cần cơ chế xử lý lỗi để tránh tài nguyên mồ côi. [DIỄN GIẢI theo H01]

Đo lường: cần phân biệt số request, số lỗi, dung lượng tệp và tài nguyên CPU/RAM. Số request dễ đếm theo tenant hơn chi phí CPU thực của từng tenant trên tiến trình dùng chung. Nếu phân bổ chi phí máy chủ theo tỷ lệ request, đó là mô hình ước tính với giả định, không phải phép đo trực tiếp CPU của từng tenant.

Định giá: gói dịch vụ có thể dựa trên chỗ ngồi, hạn mức hoặc tính năng. Giá, placement và capability là ba khái niệm khác nhau. Khả năng phê duyệt công việc

không có tính tất yếu kỹ thuật phải dùng database riêng. Nếu chỉ mở tính năng cho một gói/placement, cần gọi đúng là chính sách sản phẩm. Chưa có dữ liệu ở đây để kết luận phương án định giá sinh lời. [DIỄN GIẢI; R01 Phụ lục A, R05]

4.7 Cập nhật, khôi phục và phạm vi sự cố

Migration: thay cấu trúc dữ liệu có phiên bản. Với nhiều schema/database, cần theo dõi nơi nào đã cập nhật và nơi nào thất bại; tránh để mã mới giả định mọi tenant đã có cột mới. **Feature flag** là cờ cấu hình bật/tắt tính năng, khác với migration: tắt giao diện không hoàn tác một cột đã xóa.

Backup tạo bản sao để phục hồi; phải thử **restore** mới biết bản sao có dùng được. **RTO** là mục tiêu thời gian khôi phục, **RPO** là mục tiêu mức mất dữ liệu tính theo thời gian. Chúng là mục tiêu cần chốt và kiểm chứng, không phải con số tự có khi bật sao lưu.

PITR khôi phục tới một thời điểm bằng bản sao nền và nhật ký WAL. Cơ chế continuous archiving của PostgreSQL phục hồi cả cluster, không trực tiếp một schema hay một database trong cluster. Muốn lấy lại riêng A có thể cần khôi phục sang môi trường tạm rồi trích xuất dữ liệu phù hợp; còn tệp, quyền và các phụ thuộc cũng phải được đối chiếu. [NGUỒN; PostgreSQL: Continuous Archiving and PITR]

Chương 5

Thực nghiệm, đánh giá và trạng thái bằng chứng

5.1 Phạm vi hiện vật và phiên bản được khảo sát

[HIỆN CÓ] HEAD được kiểm tra là `301514935fe541f2ab1b3d6657a785d241fa60b1`. Có `resource/`, `meeting/`, `research/`, `report/` và `reportTemplate/`; không có cây mã `apps/`, `infra/`, `experiments/` trong checkout. Các ghi chú nghiên cứu và hình mới đã tồn tại dưới dạng chưa được Git theo dõi khi bắt đầu lượt này; trạng thái "untracked" không có nghĩa nội dung không tồn tại.

Commit `430e2d8` xóa nhiều thành phần ứng dụng và tài liệu. Snapshot dùng để đọc lịch sử là cha của commit đó: **`6dfb887aadc089df217e3a0fbe76f3c97d214913`**. Phiên này đọc bằng `git show`, không khôi phục ứng dụng vào cây làm việc, không khởi chạy hệ thống hay đo tải.

`report/config/metadata.tex` hiện còn các chỗ giữ chỗ như tên đề tài, mã và chủ nhiệm. Vì vậy, sự tồn tại của thư mục báo cáo hay PDF build không chứng minh bộ hồ sơ hiện tại đã được điền hoàn chỉnh. Không dùng kết quả kiểm tra template làm kết quả nghiên cứu. [HIỆN CÓ]

5.2 Bảng trạng thái theo loại công việc

Bảng 5.1: Tổng hợp: Bảng trạng thái theo loại công việc

Hạng mục	Đã có và phần còn thiếu
Mục tiêu/phạm vi	Có thuyết minh; còn mâu thuẫn mốc thời gian và phụ lục mở rộng chưa duyệt.
Kiến thức nền	Có tám tài liệu nghiên cứu, phân tích và hình; cần nhóm tự giải thích, rà các khẳng định có điều kiện.
Lựa chọn kiến trúc	Có đề xuất S3 và phân tích trọng số; chưa xác nhận quyết định cuối hoặc đo chứng minh ưu thế.
Ứng dụng	Có mã trong lịch sử và biên bản local; chưa kiểm runtime hiện tại.
Kiểm thử kỹ thuật	Có các lần kiểm tra khác nhau, với phiên bản và số test khác nhau; không cộng gộp thành một lần chạy.

Hạng mục	Đã có và phần còn thiếu
Thực nghiệm tải/so sánh	Có protocol cũ và thiết kế E1–E8 mới; chưa xác nhận bộ dữ liệu chính thức đủ để kết luận.
Internet/provider thật	Nguồn lịch sử vẫn nêu khoảng trống; không có xác minh mới về môi trường công khai.
Người dùng/SUS	Chưa xác nhận dữ liệu khảo sát hợp lệ; không tạo điểm hoặc số người tham gia.
Hồ sơ bàn giao	Có khung và yêu cầu đầu ra; cần kiểm nội dung, metadata, minh chứng, bản tin, tóm tắt và video theo hồ sơ.

5.3 Đối chiếu kết quả kiểm thử theo từng thời điểm

Bảng 5.2: Tổng hợp: Đối chiếu kết quả kiểm thử theo từng thời điểm

Nguồn	Nội dung ghi nhận và giới hạn
H02 — 08/09/2026	Backend 96/96, 0 skip; frontend 16 test; Playwright 3/3; application V8. Đây là checkpoint local của phần mở rộng.
H01 — ghi chú hardening 18/09	Ghi nhận thêm chức năng và migration đến V12, kiểm tra frontend và một số test backend. Không đồng nhất với bộ chạy 08/09.
H07 — 09/10/2026	Tổng 106 test, thực chạy 59, bỏ qua 47 do thiếu Docker; 0 failure/error. Maven `verify` chưa hoàn tất vì thiếu plugin offline; frontend không chạy vì thiếu Node.
Lượt tổng hợp 23/10/2026	Đọc tài liệu và một số lớp mã lịch sử; không chạy lại test ứng dụng.

H01 có bảng tổng hợp EXT ghi 106/106 và 22/22, nhưng biên bản gốc H02 ghi 96/96 và 16 test ở mốc V8. Khi trích một con số, ưu tiên nguồn của đúng lần chạy và nêu ngày/phiên bản; không sửa lịch sử bằng cách chọn con số lớn hơn. H07 cho biết các test RLS, Schema-per-tenant và nhiều test tích hợp nằm trong nhóm bị bỏ qua ở lần 09/10. Vì vậy, "0 lỗi" của lần này không tái xác nhận cô lập bằng CSDL thật.

Giới hạn kết luận: nguồn lịch sử cung cấp bằng chứng local ở những phạm vi đã ghi. Chưa đủ để tuyên bố có SLO đã đạt, triển khai Internet ổn định, hệ thống an toàn tuyệt đối hay kiến trúc tối ưu cho mọi quy mô.

5.4 Thiết kế phép đo và yêu cầu tái lập

Biến độc lập là thứ chủ động thay đổi, ở đây thường là placement. **Biến phụ thuộc** là kết quả đo, chẳng hạn độ trễ. **Biến kiểm soát** là các điều kiện giữ tương đương: bản mã, dữ liệu, cấu hình máy, tải và thời gian làm nóng. Nếu vừa đổi placement vừa đổi kích thước máy, chưa thể quy khác biệt hiệu năng cho placement.

Đơn vị lặp thường là một lần chạy được chuẩn bị lại theo giao thức. Hàng nghìn request trong cùng một run chịu chung môi trường; không nên coi tất cả là hàng nghìn thực nghiệm độc lập. Nên giữ thống kê từng tenant và từng run, nhất là trong bài toán hàng xóm ồn ào.

Manifest là bản kê môi trường và lần chạy: commit, cấu hình, seed sinh dữ liệu, thời điểm, mã kịch bản, placement, phần cứng, phiên bản và tệp đầu ra. **Checksum** giúp phát hiện tệp bị thay đổi; nó không chứng minh phương pháp đo đúng.

5.5 Bộ thí nghiệm được đề xuất

Các mục sau tóm tắt `research/06-thiet-ke-thuc-nghiem.md`. Toàn bộ là thiết kế cần chốt, không phải danh sách thí nghiệm đã chạy. Protocol 0.1 lịch sử chủ yếu cho Pool/Silo; so sánh ba placement cần phụ lục phương pháp phù hợp trước khi thu số liệu chính thức. [ĐỀ XUẤT; R06, H03]

Bảng 5.3: Tổng hợp: Bộ thí nghiệm được đề xuất

Thí nghiệm	Câu hỏi, đầu ra và điều kiện đọc kết quả
E1 — Cô lập/phân quyền	Thử host/token lệch, đổi ID, truy cập schema/database khác, tệp và job. Lưu cả yêu cầu và kiểm tra dữ liệu; phải có cấu hình cố ý làm hỏng để biết test phát hiện được lỗi.
E2 — Hàng xóm ồn ào	Giữ tải B ổn định, tăng tải A; đo độ trễ/lỗi của B. So sánh có/không biện pháp kiểm soát, theo từng run.
E3 — Cấp phát	Đo từ nhận yêu cầu tới sẵn sàng; chèn lỗi và thử lại; kiểm tài nguyên mồ côi và tenant khác.
E4 — Migration	Nâng nhiều schema/database, đo thời gian và lỗi; xác minh phiên bản từng tenant sau lỗi giữa chừng.
E5 — Khôi phục/xóa	Xuất và phục hồi A vào nơi sạch; đối chiếu dữ liệu, quyền và ảnh hưởng B. File sao lưu tồn tại chưa đủ.
E6 — Tải/năng lực	Cùng dữ liệu và hỗn hợp thao tác; đo độ trễ, thông lượng, lỗi và tài nguyên qua các mức tải.

Thí nghiệm	Câu hỏi, đầu ra và điều kiện đọc kết quả
E7 — Chi phí Bridge	Đếm nhánh phụ thuộc placement, công sức/test và chi phí router theo cách đếm được định nghĩa trước.
MB-scale	Khảo sát số lượng schema/database lớn ở tầng CSDL; là mở rộng, không tương đương thử người dùng thật.
E8 — Đo mức tiêu thụ	Tùy chọn; so sánh chi phí khi bật/tắt cơ chế đo sau khi có hiện thực.

Thử lỗi phải làm trong môi trường thí nghiệm được kiểm soát. Với E1, "tắt một chốt" không phải lúc nào cũng làm hệ thống rò dữ liệu vì lớp khác có thể vẫn chặn. Cần nêu rõ kiểm thử đang xác minh lớp riêng hay toàn hệ thống, và xác nhận đột biến đã thực sự tạo điều kiện mà test được thiết kế để bắt. Không coi một đột biến không có tác động là bằng chứng test yếu hoặc mạnh.

5.6 Định nghĩa và cách diễn giải các chỉ số

Bảng 5.4: Tổng hợp: Định nghĩa và cách diễn giải các chỉ số

Chỉ số	Cách hiểu
p95 độ trễ	Khoảng 95% quan sát không vượt giá trị này, theo cách tính percentile đã chọn; cần ghi loại request và phạm vi gom.
Thông lượng	Số thao tác/request hoàn tất mỗi đơn vị thời gian; ghi rõ có tính lỗi hay không.
Tỷ lệ lỗi	Số lỗi chia số yêu cầu trong phạm vi định nghĩa; tách lỗi kỳ vọng của test trái phép khỏi lỗi tải bình thường.
IR	p95 của tenant nạn nhân khi có tải gây nhiễu chia p95 khi không có; hai điều kiện phải tương đương.
CPU/RAM/kết nối	Cho thấy tài nguyên và điểm nghẽn; chưa tự động giải thích nguyên nhân nếu thiếu tương quan thời gian và cấu hình.

Ví dụ minh họa: p95 của B tăng từ 200 ms lên 500 ms thì IR = 2,5. Đây là số giả định để giải nghĩa tỷ số, không phải số đo của đề tài. Cần xem cả lỗi: hệ thống có thể trả lỗi nhanh và tạo p95 đẹp giả tạo nếu chỉ nhìn request thành công.

Điểm cần chỉnh trước khi khóa phương pháp: R06 có cách diễn giải khoảng tin cậy chứa 1 như cơ sở giữ giả thuyết "không đổi cô lập hiệu năng". Cách viết thận trọng

là **chưa đủ bằng chứng khác biệt**, không phải đã chứng minh tương đương. Muốn kết luận không kém hơn hoặc tương đương, cần đặt biên chấp nhận và phương pháp đánh giá tương ứng trước. Các ngưỡng 30 giây, 15 phút, 1,5 lần, 10 phút ở R06 cũng vẫn là đề xuất, không phải cam kết đã được phê duyệt.

5.7 Mối đe dọa tính hợp lệ

Tải tổng hợp có thể khác hành vi thật; VPS chung có nhiễu; số run ít làm ước lượng chưa ổn định; cấu hình ưu ái một placement có thể gây sai lệch; bộ test chỉ bao một phần đường truy cập. Kết luận từ một ứng dụng và một cấu hình không tự khái quát sang mọi SaaS. Cần báo cáo run hỏng, lý do loại và dữ liệu thiếu thay vì chỉ giữ lượt đẹp.

Chương 6

Kết luận hiện tại và kế hoạch hoàn thiện

6.1 Kết quả tổng hợp và giới hạn kết luận

Qua tổng hợp các nguồn, nhóm đã hệ thống hóa khái niệm, phân tích các kịch bản chia sẻ tài nguyên và có bản ứng dụng hỗ trợ ba placement trong lịch sử mã nguồn. Có bằng chứng kiểm thử local ở các mốc riêng biệt. Bộ phân tích mới đề xuất Schema-per-tenant làm kiến trúc chính với các giả định về quy mô và yêu cầu. Nhóm còn phải chốt phạm vi, đồng bộ lựa chọn với hiện thực, hoàn thiện giao thức và thu bằng chứng thực nghiệm trước khi kết luận về hiệu năng, vận hành và ưu thế kiến trúc.

Đây là kết luận tổng hợp từ các nguồn đã đọc, không phải xác nhận nghiệm thu. Việc thiếu dữ liệu không xóa giá trị của mô hình và cơ chế đã xây dựng; nó giới hạn loại kết luận được phép rút ra.

6.2 Các vấn đề cần xác nhận

Bảng 6.1: Tổng hợp: Các vấn đề cần xác nhận

Quyết định	Vì sao cần và đầu ra mong đợi
Thời hạn/phạm vi chính thức	Đối chiếu hồ sơ đã ký, giải quyết mốc 5–10 và 3–8; ghi văn bản căn cứ.
S3 hay Bridge là trọng tâm	Chốt một phát biểu mục tiêu, vai trò đối chứng và tiêu chí đổi lựa chọn.
Quy mô thiết kế	Nêu số/loại tenant mục tiêu và căn cứ; phân biệt với 3–5 tenant thực nghiệm.
Phạm vi mở rộng	Chốt tình trạng Phụ lục A, Silo, tùy biến và MB-scale; không tự nâng đề xuất thành yêu cầu đã duyệt.
Bản mã/môi trường dùng tiếp	Chọn snapshot để rà và chạy; không giả định checkout tài liệu đang là ứng dụng sẵn chạy.
Giao thức và ngưỡng	Chốt workload, seed, số run, môi trường, định nghĩa metric, điều kiện loại và cách thống kê.

6.3 Lộ trình hoàn thiện và tiêu chí đầu ra

Bước 1 — Đồng bộ hiểu biết. Mỗi thành viên tự giải thích tenant, shared/dedicated, ba placement và luồng yêu cầu A/B; người nghe hỏi một trường hợp khác để tránh học thuộc. Hoàn thành khi lời giải đúng có dẫn chứng và người học nhận ra giới hạn của nó.

Bước 2 — Chốt hồ sơ nghiên cứu. Nhóm lập biên bản cho các quyết định ở bảng trên; chỉ rõ ai xác nhận và căn cứ nào. Hoàn thành khi câu hỏi nghiên cứu, phạm vi, ứng viên và ngưỡng không mâu thuẫn nhau. Tài liệu học không thay bước này.

Bước 3 — Rà bản ứng dụng đã chọn. Đối chiếu cơ chế của từng placement, credential, nhóm kết nối, job/tệp và ma trận quyền. Chạy lại kiểm tra local phù hợp trong môi trường có đủ phụ thuộc. Hoàn thành khi có log mới, tổng số chạy/skip rõ và các lỗi cản trở được xử lý. Đây là việc tiếp theo, chưa thực hiện trong lượt viết tài liệu này.

Bước 4 — Diễn tập phương pháp. Kiểm E1, E3, E6 ở quy mô nhỏ để biết công cụ có hoạt động; kiểm manifest, dữ liệu thô và tiêu chí run hợp lệ. Hoàn thành khi người khác có thể lặp lại; dữ liệu diễn tập được tách khỏi dữ liệu chính thức.

Bước 5 — Thu và phân tích dữ liệu chính thức. Thực hiện khi phạm vi/giao thức đã được xác nhận; báo cáo kết quả từng run, từng tenant và giới hạn. Hoàn thành khi mọi kết luận có đầu ra truy vết được, không chỉ có bảng số chép tay.

Bước 6 — Hoàn thiện hồ sơ. Viết lại kết luận theo bằng chứng, đồng bộ metadata, tóm tắt, bản tin và minh chứng sản phẩm; xem trực tiếp PDF. Kiểm tra hình thức thành công chỉ chứng minh chất lượng tài liệu, không thay chứng cứ khoa học.

Không gán tên người phụ trách hay ngày hẹn mới trong tài liệu vì chưa có quyết định phân công hiện tại. Có thể dùng bảng công việc với các cột "việc, phụ trách, đầu vào, đầu ra, điều kiện xong, hạn được thống nhất".

6.4 Tổ chức chuyển giao kiến thức trong nhóm

Bảng 6.2: Tổng hợp: Tổ chức chuyển giao kiến thức trong nhóm

Buổi	Hoạt động và sản phẩm học tập
1 — Ngôn ngữ chung	Đọc Chương 1–2; vẽ A/B trên ba tầng; giải thích tenant khác user/database.
2 — Chọn kiến trúc	Đọc Chương 3; mỗi người bảo vệ một phương án rồi nêu điều kiện khiến mình đổi ý.

Buổi	Hoạt động và sản phẩm học tập
3 — Lần theo mã	Đọc Chương 4 và H04–H06; ghi điểm đặt/xóa ngữ cảnh, nơi kiểm quyền và đường worker.
4 — Hiểu bằng chứng	Đọc Chương 5; phân loại test cũ, kết quả chấm điểm, giả thuyết và phép đo còn thiếu.
5 — Diễn tập trình bày	Mỗi người trình bày một chuỗi nhu cầu → cơ chế → bằng chứng → giới hạn, rồi trả lời tình huống mới.

Mẫu ghi học tập: ngày; người học; câu hỏi; câu trả lời ban đầu; căn cứ dùng; chỗ thiếu; giải thích bổ sung; câu hỏi kiểm lại khác; kết quả kiểm lại. Không ghi "đã hiểu" chỉ vì người học lặp lại đáp án ngay sau khi được nhắc.

6.5 Đóng góp có thể hướng tới và giới hạn

Đóng góp khả thi của đề tài là một cách mô hình hóa rõ ràng, lựa chọn kiến trúc có căn cứ trong phạm vi xác định, hiện thực minh họa và bằng chứng kiểm chứng tái lập được. Việc ghép các công nghệ phổ biến hoặc có nhiều tính năng chưa tự tạo ra tính mới. Muốn tuyên bố đóng góp mới, cần đối chiếu công trình liên quan và chỉ ra phần khác biệt có ý nghĩa.

Các phần về Base.vn, Salesforce và một số tài liệu học thuật còn thiếu kiểm chứng trong bộ nguồn hiện tại. Không suy đoán kiến trúc nội bộ của sản phẩm chỉ từ giao diện hoặc thông tin tiếp thị. Chỉ dùng phát biểu nhà cung cấp thực sự công bố và ghi rõ mức chứng cứ. [HIỆN CÓ; R07 và README nghiên cứu]

Phụ lục A. Câu hỏi ôn tập và gợi ý trả lời

Câu hỏi Chương 1

1. Với ví dụ A và B, tại sao `user_id` chưa đủ để xác định dữ liệu được phép đọc?
2. Tại sao 3–5 thuê bao trong thực nghiệm chưa chứng minh hệ thống phục vụ tốt vài trăm thuê bao?
3. Kể một kết quả thuộc tính năng ứng dụng và một kết luận kiến trúc cần thêm bằng chứng.

Câu hỏi Chương 2

1. Một API phục vụ A/B, hai database cùng PostgreSQL: tầng nào chung, tầng nào riêng?
2. Hai bản sao API cùng phục vụ A/B có phải là hai tenant không?
3. Vì sao mặt phẳng điều khiển phải biết placement của A dù nó phục vụ toàn hệ thống?

Câu hỏi Chương 3

1. S3 được điểm cao trong mô hình có đồng nghĩa S3 chạy nhanh nhất không?
2. Tại sao cách dùng tài khoản CSDL ảnh hưởng cả bảo mật lẫn số nhóm kết nối?
3. Nêu một điều kiện có thể làm nhóm đổi đề xuất S3 mà không cần phủ nhận toàn bộ lý thuyết trước đó.

Câu hỏi Chương 4

1. Tại sao cần test task của B ngay cả khi host và token đều là A?
2. Tại sao `search_path=tenant_a` không tự bảo vệ `tenant_b.task`?
3. Worker gửi xong rồi chết có thể gây vấn đề gì? Outbox còn cần cơ chế nào?
4. Silo database trên cùng cluster có cho PITR riêng từng thuê bao một cách trực tiếp không?

Câu hỏi Chương 5

1. “106 test, 0 failure, 47 skip” khác “106 test chạy đạt” ở đâu?
2. Nếu p95 chung đẹp nhưng B chậm khi A tải lớn, kết luận nào còn thiếu?

3. Khoảng tin cậy của một tỷ số chứa 1 có chứng minh hai kiến trúc tương đương không?
4. Vì sao lưu ảnh dashboard chưa đủ để người khác lặp lại kết quả?

Gợi ý trả lời

Chương 1: một user có thể ở nhiều tenant với quyền khác nhau; cần biết tenant và hành động. Mẫu 3–5 tenant chỉ kiểm một quy mô, chưa phản ánh vận hành hàng trăm schema/kết nối. Trang tạo task là tính năng; chống đọc chéo hoặc chịu tải cần bộ kiểm chứng tương ứng.

Chương 2: API và máy chủ CSDL chung, database riêng; nhiều replica không tạo tenant mới. Mặt phẳng điều khiển phải lưu mapping tenant–placement để cấp phát và định tuyến; phạm vi quản trị toàn hệ thống không xóa ngữ cảnh tài nguyên.

Chương 3: điểm chuyên gia là đầu vào quyết định, không phải benchmark. Credential riêng thu hẹp quyền nhưng có thể cần pool riêng; login chung thay đổi mô hình đe dọa. Các yêu cầu khôi phục, quy mô, chi phí migration hoặc bằng chứng lỗi cô lập có thể làm thay đổi lựa chọn.

Chương 4: host/token đúng không chứng minh ID thuộc đúng tenant/dự án. `search_path` chọn tên mặc định, quyền quyết định truy cập. Worker có thể lặp hành động nên cần chống trùng/retry/kiểm lại trạng thái. PITR chuẩn tác động cả cluster; cần quy trình khác để trích phục hồi một thuê bao.

Chương 5: chỉ 59 test thực chạy trong ví dụ đã dẫn. Chỉ số chung có thể che tenant bị ảnh hưởng; cần đo theo tenant. Chưa thấy khác biệt không phải chứng minh tương đương. Dashboard thiếu phiên bản, cấu hình, dữ liệu thô và quyết định lọc để tái lập.

Tình huống vận dụng

1. Giữ nguyên tài khoản và tenant nhưng đổi dự án sang dự án không được mời. Lớp kiểm tra nào phải từ chối?
2. Một kết nối vừa rollback yêu cầu A rồi phục vụ B. Cần quan sát gì để phát hiện ngữ cảnh còn sót?
3. API chung chết trong mô hình database riêng. Những tenant nào có thể bị ảnh hưởng và tại sao?
4. Bộ test trả 403 cho mọi request, cả hợp lệ lẫn trái phép. Có thể kết luận cô lập đạt không?
5. Phương án nhanh hơn nhưng một lần đọc được dữ liệu tenant khác. Có nên giữ vì

điểm tổng cao hơn không?

Gợi ý: quyền dự án; trạng thái giao dịch/ngữ cảnh và dữ liệu thấy được; mọi tenant dùng API đó; cần đối chứng truy cập hợp lệ để tránh “đúng do từ chối tất cả”; vi phạm cô lập là điều kiện loại.

Phụ lục B. Danh mục nguồn và truy vết bằng chứng

Nguồn hiện có trong repository

Các đường dẫn dưới đây tính từ file Markdown này. Trong PDF, dùng mã nguồn và đường dẫn để tra repository; các liên kết file tương đối thuận tiện nhất khi đọc bản Markdown tại chỗ.

Bảng 6.3: Tổng hợp: Nguồn hiện có trong repository

Mã	Nguồn và phạm vi sử dụng
R01	Thuyết minh: mục tiêu, phạm vi, tiến độ, sản phẩm, Phụ lục A chưa duyệt.
R02	Research Report PDF: đề cương sáu chương và ghi chú của nhóm.
R03	Ghi chú định hướng, biên bản họp: cách tiếp cận và việc cần làm.
R04	Khái niệm, mô hình, ngăn xếp: nền khái niệm.
R05	Lựa chọn kiến trúc, kết quả phân tích: đề xuất S3 và giả định.
R06	Kịch bản, thiết kế thực nghiệm, cập nhật: đánh đổi, kế hoạch và vấn đề mở.
R07	Sổ nguồn: căn cứ và đính chính trước đó. Việc sổ nguồn ghi "đã đọc" thuộc lần nghiên cứu trước; tài liệu này không nhận là đã tự đọc lại mọi nguồn.

Các nguồn PostgreSQL và Microsoft liên kết tại phần nội dung được mở trực tiếp trong lượt tổng hợp 23/10/2026. Nguồn Kabbedijk/AWS/ISO được dẫn qua bộ nghiên cứu nội bộ khi đã ghi rõ; không bổ sung DOI, trích dẫn nguyên văn hay xác nhận nguồn chưa thực hiện.

Nguồn lịch sử Git

Tất cả mã H dưới đây dùng snapshot `6dfb887aadc089df217e3a0fbe76f3c97d214913`. Các tệp hiện không có sẵn trong checkout. Với H04–H06, đường dẫn gốc là `apps/api/src/main/java/`, tiếp đến gói Java `vn.edu.ctu.saas`; tên thư mục con và lớp được ghi trong bảng.

Bảng 6.4: Tổng hợp: Nguồn lịch sử Git

Mã	Tệp và phần đối chiếu
H01	`docs/PROJECT_STATUS.md`: trạng thái, hardening, phần đã hiện thực và giới hạn kiểm chứng.
H02	`docs/testing/extension-local-2026-09-08.md`: kiểm tra ba placement ở checkpoint V8.
H03	`docs/research/protocol.md`: phiên bản 0.1 và yêu cầu bổ sung phương pháp khi so sánh ba placement.
H04	Thư mục `tenant`, lớp `DefaultTenantDataSourceResolver.java`: `resolve`, `schemaName`, cấu hình pool.
H05	Thư mục `tenant`, lớp `TenantJdbcExecutor.java`: `execute`, `setLocal`.
H06	Thư mục `security`, lớp `TenantContextFilter.java`: `establish`, dọn context trong `finally`.
H07	`docs/testing/preliminary-report-2026-10-09.json`: tổng test, executed, skipped và giới hạn môi trường.

Ví dụ đọc lại bằng lệnh chỉ đọc, chạy tại gốc repository:

```
git show 6dfb887:docs/PROJECT_STATUS.md
git show 6dfb887:docs/testing/preliminary-report-2026-10-09.json
git show 6dfb887:docs/research/protocol.md
```

File `source-manifest.json` cạnh tài liệu lưu commit và mã băm các đầu vào để xác định snapshot đã dùng. Nó không chứa bí mật hay dữ liệu môi trường ứng dụng. Dựng lại PDF bằng `bash research/learning/build.sh` từ gốc repository; đây chỉ là lệnh dựng tài liệu.